

Chapter 6: Spark Framework

❖ Topic covered:

- Introduction to GPU Computing
- CUDA Programming Model, CUDA API, Simple Matrix Multiplication in CUDA
- CUDA Memory Model
- Shared Memory Matrix Multiplication
- Additional CUDA API Features.

❖ Spark Framework:

➤ What is Spark framework in Python?

- PySpark is the Python API for Apache Spark. It enables you to perform real-time, large-scale data processing in a distributed environment using Python. It also provides a PySpark shell for interactively analyzing your data.

➤ Why use Spark in Python?

- PySpark brings robust and cost-effective ways to run machine learning applications on billions and trillions of data on distributed clusters 100 times faster than the traditional python applications. PySpark has been used by many organizations like Amazon, Walmart, Trivago, Sanofi, Runtastic, and many more.

➤ Key Features of Apache Spark:

- In-Memory Processing: Spark processes data in-memory, making it significantly faster than traditional disk-based systems like Hadoop's MapReduce.
- Ease of Use: Spark provides high-level APIs in Scala, Java, Python, and R, making it accessible to a wide range of developers.
- Sophisticated Analytics: Spark includes libraries for machine learning (MLlib), graph analysis (GraphX), and stream processing (Spark Streaming).
- General-Purpose Cluster Computing: Spark is not limited to batch processing; it can handle various workloads efficiently.

➤ Components of the Spark Ecosystem:

- Apache Spark Core:
 - The underlying execution engine for Spark.
 - Supports in-memory computing and external storage structures.
- Spark SQL:
 - Operates with structured data.
 - Provides interfaces for both computation and data information.
- Spark Streaming:
 - Processes real-time streaming data from sources like HDFS, Flume, and Kafka.
 - Enables complex algorithms and live dashboard updates.
- Machine Learning Library (MLlib):
 - Rich library of machine learning algorithms (clustering, regression, classification, etc.).
 - Supports model tuning, evaluation, and ML pipelines.
- GraphX:
 - Manipulates graph databases and performs graph computations.
 - Combines ETL processes, graph computation, and exploratory analysis.
- **Introduction to GPU Computing:**
 - GPU stands for Graphics Processing Unit. GPUs are also known as video cards or graphics cards. In order to display pictures, videos, and 2D or 3D animations, each device uses a GPU.
 - A GPU performs fast calculations of arithmetic and frees up the CPU to do different things.

❖ **CUDA Programming Model, CUDA API, Simple Matrix Multiplication in CUDA:**

- CUDA (Compute Unified Device Architecture) is an open-source parallel computing platform developed by NVIDIA.

- It allows software to utilize certain types of graphics processing units (GPUs) for accelerated general-purpose processing (GPGPU).
- CUDA enables developers to speed up compute-intensive applications by leveraging the parallel compute engine in NVIDIA GPUs.

❖ **The CUDA Programming Model:**

- The CUDA programming model provides an abstraction of GPU architecture, bridging the gap between applications and GPU hardware.
- Key concepts:
 - Thread-Blocks: The smallest group of threads allowed by the programming model.
 - Grids: Arrangements of multiple thread-blocks.
 - 1-D, 2-D, or 3-D Arrangements: Depending on the problem, thread-blocks and grids can be organized in different dimensions.
 - Thread-Level Parallelism: Each thread performs a specific computation.
 - Vector-Vector Multiplication: Each element in the output matrix is calculated by a separate CUDA thread.

❖ **Introduction to GPU Computing**

- GPU stands for Graphics Processing Unit. GPUs are also known as video cards or graphics cards. In order to display pictures, videos, and 2D or 3D animations, each device uses a GPU.
- A GPU performs fast calculations of arithmetic and frees up the CPU to do different things.

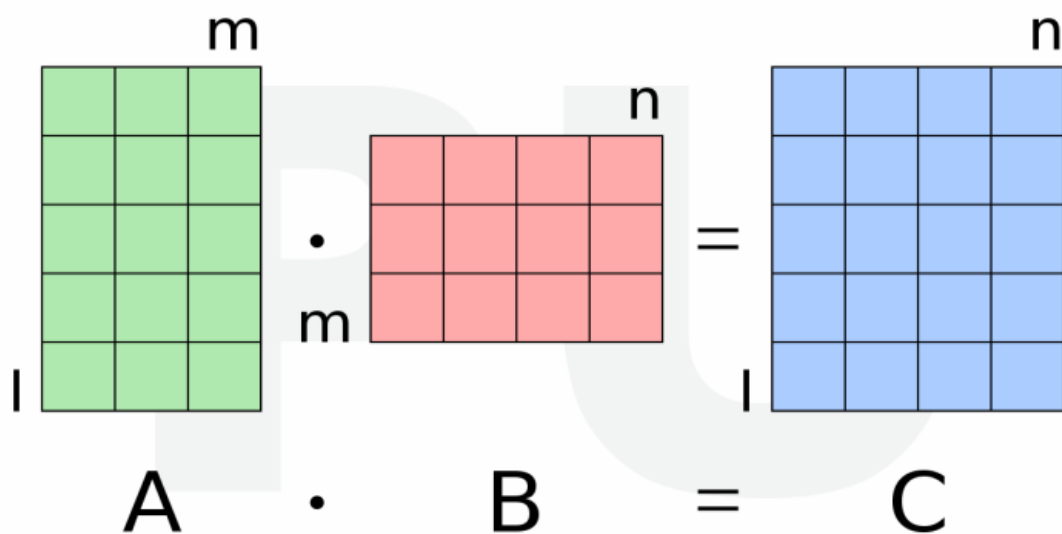
❖ **CUDA API:**

- CUDA (or Compute Unified Device Architecture) is a parallel computing platform and application programming interface (API) that allows software to use certain types of graphics processing units (GPUs) for general

purpose processing, an approach called general-purpose computing on GPUs (GPGPU).

❖ **Simple Matrix Multiplication in CUDA:**

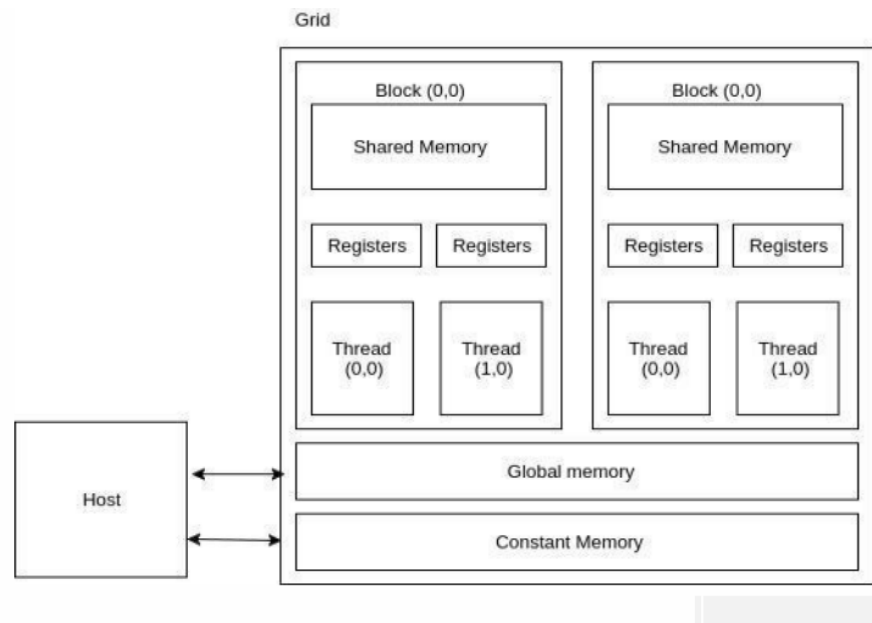
- Each thread loads one row of matrix A and one column of matrix B from global memory, do the inner product, and store the result back to matrix C in the global memory.
- In the naive implementation, the amount of computation is $2 \times M \times N \times K$ flop, while the amount of global memory access is $2 \times M \times N \times K$ word.
- In mathematics, particularly in linear algebra, matrix multiplication is a binary operation that produces a matrix from two matrices. For matrix multiplication, the number of columns in the first matrix must be equal to the number of rows in the second matrix.



❖ **CUDA Memory Model:**

- It is used for storing data that will not change over the course of kernel execution.

- It supports short-latency, high-bandwidth, read-only access by the device when all threads simultaneously access the same location.
- There is a total of 64K constant memory on a CUDA capable device. The constant memory is cached.



❖ **What are the different types of CUDA memory?**

- A CUDA device has a number of different memory components that are available to programmers - register, shared memory, local memory, global memory and constant memory.
- Figure 6 illustrates how threads in the CUDA device can access the different memory components.
- In CUDA only threads and the host can access memory.

❖ **Shared Memory Matrix Multiplication:**

- Shared memory matrix multiplication is particularly useful in GPU programming, where you have a large number of threads that can work in parallel. In this context, shared memory can significantly reduce the need to access data from slower global memory, resulting in a performance boost.
- The specific implementation of shared memory matrix multiplication can vary depending on the programming language and the parallel computing framework you're using, such as CUDA for NVIDIA GPUs or OpenCL for more general-purpose parallel computing. The key is to load data into shared memory, ensure proper synchronization, and take advantage of parallelism for the matrix multiplication step.

❖ **Additional CUDA API Features:**

1. cuDF and cuML:

- cuDF: A GPU-accelerated DataFrame library that mimics the popular Pandas library. It allows data scientists and analysts to work with structured data using familiar Pandas-like syntax while leveraging GPU acceleration across the data pipeline.
- cuML: A machine learning library for GPUs. It provides a rich set of machine learning algorithms (such as clustering, regression, classification) that can be accelerated using CUDA. cuML enables end-to-end GPU-accelerated machine learning workflows.

2. High-Performance Data Analytics:

- NVIDIA's RAPIDS open-source software suite, combined with CUDA, accelerates analytics pipelines on NVIDIA GPUs. It reduces data analytics operations like data loading, preprocessing, and model training. RAPIDS includes libraries like cuDF, cuML, and cuGraph for various data analytics tasks.

3. Parallel Programming for GPUs:

- CUDA is a parallel computing platform and programming model developed by NVIDIA. It allows developers to harness the power of GPUs for general-purpose parallel computing.
- Beyond matrix multiplication, CUDA can accelerate a wide range of compute-intensive applications, including machine learning, scientific simulations, and data analytics.

4. Graph Analytics with cuGraph:

- cuGraph is a GPU-accelerated graph analytics library. It provides algorithms for analyzing large-scale graphs and networks.
- Use cuGraph to perform tasks like community detection, centrality analysis, and graph traversal efficiently on GPUs.